

Multitable Clustering File Organization

Concept

Key Idea

- **Multitable clustering** is a file organization technique where multiple relations are stored in the same file.
- Tuples from different relations that are **often queried together** are clustered in the same disk block.
- This reduces the **disk I/O cost** for queries involving joins across relations.

Motivation

- Normally, each relation is stored in a separate file.
- Join queries require scanning multiple files, causing high disk I/O.
- **Clustering related tuples together** (e.g., primary key + foreign key) minimizes block accesses.

Organization

- A disk block may hold tuples from **two or more relations**.
- Example: Storing **Student** and **Enrollment** together.
- This ensures queries like “Find all courses taken by a student” are answered faster.

Example

Illustrative Example

Relations:

- Student(SID, Name, Dept)
- Enrollment(SID, CourseID, Grade)

Clustering Strategy:

- In normal storage: each relation is kept separately → more joins, more I/O.
- In clustering: store each student's record **together with all their enrollment records** in the same block.
- Example query:

“List all courses taken by Srinivasan.”

Normal File Organization (Separate Tables)

SID	Name	Dept
101	Srinivasan	CS
102	Rahman	EE

SID	CourseID	Grade
101	CSE101	A
101	CSE102	B+
102	EE201	A-
102	EE202	B

Table 1: Normal storage: Student and Enrollment kept separately

Multitable Clustering File Organization

Advantages

- **Faster joins** for clustered relations.
- **Reduced disk I/O** for queries that often use multiple relations.
- Efficient for queries like “Find all enrollments of a student.”

Block 1	
Student:	(SID=101, Name=Srinivasan, Dept=CS)
Enrollment:	(SID=101, CourseID=CSE101, Grade=A)
Enrollment:	(SID=101, CourseID=CSE102, Grade=B+)

Block 2	
Student:	(SID=102, Name=Rahman, Dept=EE)
Enrollment:	(SID=102, CourseID=EE201, Grade=A-)
Enrollment:	(SID=102, CourseID=EE202, Grade=B)

Table 2: Clustering: Student and Enrollment stored together in blocks

Disadvantages

- Insertions/updates are more complex (finding space in blocks).
- Space may be wasted if one relation grows faster than the other.
- Not useful when relations are rarely queried together.

Comparison with Separate File Organization

Multitable Clustering	Separate File Storage
Multiple relations stored in one file	Each relation stored in a separate file
Optimized for join queries	Optimized for single-relation queries
Reduces disk I/O by clustering related tuples	Higher I/O for frequent joins
Complex updates and maintenance	Easier updates and inserts